arXiv:2305.06404v1 [cs.CL] 10 May 2023

LACoS-BLOOM: Low-rank Adaptation with Contrastive objective on 8 bits Siamese-BLOOM

Wen-Yu Hua[1,*], Brian Williams[2] and Davood Shamsi[2]

[1]Seattle, Apple, USA
[2]Cupertino, Apple, USA

Abstract

Text embeddings are useful features for several NLP applications, such as sentence similarity, text clustering, and semantic search. In this paper, we present a Low-rank Adaptation with a Contrastive objective on top of 8-bit Siamese-BLOOM, a multilingual large language model optimized to produce semantically meaningful word embeddings. The innovation is threefold. First, we cast BLOOM weights to 8-bit values. Second, we fine-tune BLOOM with a scalable adapter (LoRA) and 8-bit Adam optimizer for sentence similarity classification. Third, we apply a Siamese architecture on BLOOM model with a contrastive objective to ease the multi-lingual labeled data scarcity. The experiment results show the quality of learned embeddings from LACoS-BLOOM is proportional to the number of model parameters and the amount of unlabeled training data. With the parameter efficient fine-tuning design, we are able to run BLOOM 7.1 billion parameters end-to-end on a single GPU machine with 32GB memory. Compared to previous solution Sentence-BERT, we achieve significant improvement on both English and multi-lingual STS tasks.

Keywords

Parameter efficient fine-tuning, large language model, multilingual semantic similarity embeddings

1. Introduction

Large Language Models (LLMs) are capable of generating human-like language and can be utilized for a wide range of applications, including question answering, summarization, and more. The performance of natural language tasks typically improves as the scale of the model increases [1]. Therefore, modern language models have hundreds of billions of parameters [2, 3, 4]. Any mention of LLMs is likely to spark discussion around decoder-only Transformer models, where the objective is to predict the next token in a sequence [5, 6, 2]. However, the text embedding model is equally important. Text representation, also known as text embedding, is the output of an encoder-based Transformer [7, 8]. It is designed to capture the meaning of texts that can be applied to downstream tasks, such as retrieval and clustering. Sentence-BERT [9] is a classic model for generating similar text representations. It is built on top of BERT [7] and then applied with a Siamese architecture on sentence pairs to classify if a pair is paraphrase identical. As a result, similar context words will have closer embedding representations. Although

ReNeuIR'23: Workshop on Reaching Efficiency in Neural Information Retrieval, July 23–27, 2023, Taipei, Taiwan
*Corresponding author.
wenyu_hua@apple.com (W. Hua); brian_d_williams@apple.com (B. Williams); davood@apple.com (D. Shamsi)
© 2023 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).
CEUR Workshop Proceedings (CEUR-WS.org)

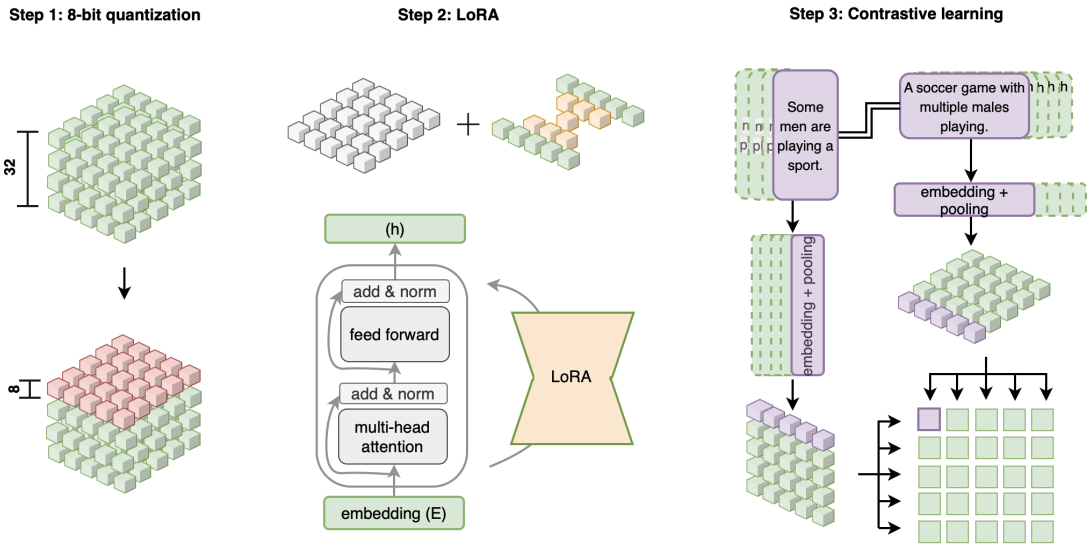

Figure 1: LACoS-BLOOM Design. Consider a group of green cubes representing a transformer module. In Step 1, we first quantize the model parameters from 32 float points (green cubes) into 8-bit integers (red cubes). In Step 2, we fine-tune the model by freezing the parameters (gray cubes) and enabling only less than 1% of the adapters (green and orange cubes), where the orange cubes represent the number of adapters to tune. In Step 3, we only use the entailment class premise and hypothesis pairs from the NLI datasets for training, and we apply a Siamese architecture with MNR contrastive objective to improve performance (the purple cube represents the positive pair, while the others represent negative pairs.)

Sentence-BERT has been successful in several applications in both industry and academia, it only supports English and is a relatively small model.

BigScience Large Open-science Open-access Multilingual Language Model (BLOOM) was released in 2022 [10] and it was trained from 46 natural languages and 13 programming languages. The training datasets cover many research questions surrounding advanced topics such as capabilities, limitations, potential improvements, bias, ethics, environmental impact, and the general AI cognitive research landscape [11]. To the best of our knowledge (Feb. 2023), BLOOM is the largest publicly available LLM in natural language processing (NLP). The largest BLOOM has 176 billion parameters. BLOOM is powerful, but it is an autoregressive language model aimed at natural language generation. Although it has achieved state-of-the-art (SOTA) performance on several unsupervised NLP tasks, for domain-specific tasks, such as generating semantically meaningful representations, we still need to fine-tune the pre-trained LLM. Our initial attempt is to fine-tune BLOOM with Siamese architecture. However, BLOOM is trained with large-scale parameters on a cluster with hundreds of GPUs, which is less realistic for many situations. In addition, well-performing text embeddings normally require a large amount of labeled data, which is another limitation as the usefulness of multi-lingual labeled data is scarce and expensive.

To overcome the challenges, we propose a parameter efficient fine-tuning solution, i.e., Low-rank Adaptation with a Contrastive objective on top of 8-bit Siamese-BLOOM (LACoS-BLOOM). We take inspiration from the work of bitsandbytes [12], where the model weights are frozen in 8-bit format (a model with 7.1 billion parameters is reduced from 20Gb down to 6Gb). We

then fine-tune BLOOM with less than 1% of the parameters using Low-Rank Adaptation (LoRA) [13] and update the weights with an efficient 8-bit Adam optimizer [14]. Lastly, to make the representation semantically meaningful, we train the model on single-class samples with a multiple negative ranking (MNR) objective on a Siamese architecture [15, 16]. With the design of LACoS, we are able to run various BLOOMs into a single GPU (BLOOM model parameters from 560 million (560m) to 7.1 billion (7b1)). On the evaluation semantic textual similarity (STS) tasks, we achieved significant improvements over the baseline Sentence-BERT.

Next, we present the LACoS-BLOOM model in Section 2. This is followed by the experimental set up and results in Section 3. The related work is in Section 4. Finally we conclude the work and discuss the next step in Section 5.

2. Model

LACoS-BLOOM (Fig. 1) is a text embedding model that generates semantically meaningful representations for multilingual texts. Several techniques have been applied to make LACoS-BLOOM more practical with fewer computational resources and to produce high-quality representations. This includes quantizing the large number of model weights using 8-bit block-wise quantization. The model is fine-tuned using a scalable LoRA and 8-bit Adam optimizer. Finally, the model is enhanced by a Siamese network with a MNR loss.

2.1. 8-bit block-wise quantization

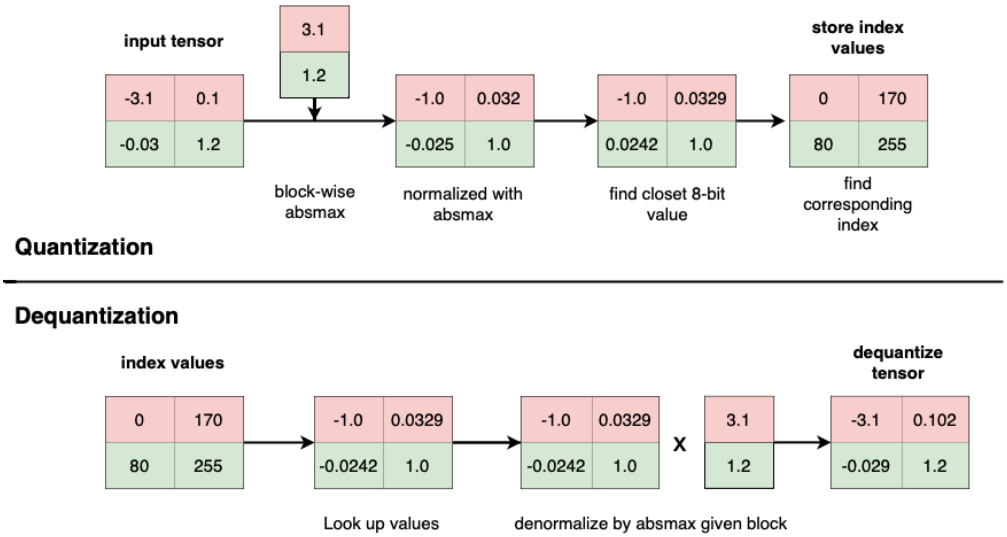

Figure 2: Block-wise quantization and dequantization with block $B = 2$ (red and green blocks)

We use 8-bit block-wise quantization from [12]. Figure 2 illustrates the steps. A 2×2 matrix split by block size $B = 2$ (red and green blocks). Within each block, we find the absolute maximum values, and then use these values to map the float32 weights to 8-bit integer

values/indices. Once the weights have been quantized, the indices are stored, which can significantly reduce the footprint of the model. When we update the model, those parameters are de-quantized back to 32 or 16 float points for just-in-time multiplication. The method we use differs from other 8-bit approaches in that we use 8-bit quantization only for storage and perform computations in float16 or float32. This allows the use of nonlinear quantization that is tailored to the distribution of each individual weight, which can reduce error without affecting inference performance.

2.2. Low-Rank adaptation

The adapter approach utilizes small, trainable matrices with low rank to approximate weight updates for downstream tasks. An approach, LoRA, represents updates using a low-rank decomposition in Eq. (1) of the pre-trained weight matrics:

$$\mathbf{W} + \Delta W = \mathbf{W} + \mathbf{W}_{\text{down}} \times \mathbf{W}_{\text{up}}. \tag{1}$$

The decomposition in Eq. (1) is represented by two tunable matrices, $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d \times r}$ and $\mathbf{W}_{\text{up}} \in \mathbb{R}^{r \times k}$, ($r \ll \min(d, k)$), and is applied to query and value projection matrices in multi-head attention layers from [13]. In this work, we apply LoRA to feed-forward linear layers and the last hidden embedding layer, as suggested by previous research [17].

2.3. Siamese network

The use of LoRA on LLMs has been successful for fine-tuning domain-specific tasks, but another limitation for finetuning tasks is the availability of labeled data. To address this issue, we propose using a Siamese architecture with contrastive objective, i.e., MNR loss [18].

MNR loss with Siamese architecture is an approach that allows the model to learn accurate semantic similarity representations despite the limitations of limited labeled data. Given a sequence of mini-batch size n, $P = (u_1, v_1), (u_2, v_2), ..., (u_n, v_n)$, where (u_i, v_i) is a positive pair, and (u_i, v_j) for $i \neq j$ are negative pairs. Sentence pairs are passed through LoRA 8-bit BLOOM to obtain the last hidden layer embedding for each token. A mean pooling layer is then applied to produce sentence-level embedding. The similarity score between the embedding pair (u, v) is computed by a cosine function and denoted as $sim(u, v)$. Note that given each mini-batch, there is only 1 positive pair, and others are negatives (denoted $\bar{P}$) (Step 3 in Fig. 1). The goal is to minimize the negative log-likelihood for softmax-normalized scores in Eq (2):

$$L = \sum_{(u,v) \in P} log \frac{exp(sim(u, v))}{exp(sim(u, v)) + \sum_{w \in \bar{p}} exp(sim(u, w)}. \tag{2}$$

3. Experiment

3.1. Experimental setup

We perform two experiments to train the LACoS-BLOOM model. The first experiment is the Stanford Natural Language Inference (SNLI) [19] and Multi-Genre NLI (MNLI) [20] datasets,

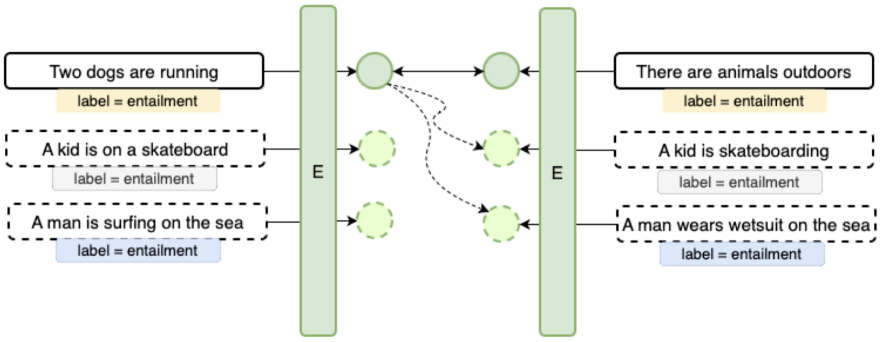

Figure 3: Siamese network with only entailment pair of samples in NLI data; where the solid line shows a positive pair and dashed lines show negative samples given a mini-batch

while the second is the Multilingual NLI (multi-NLI) dataset [21]. During training, we only employ data pairs belonging to the entailment class and apply a Siamese network with MNR loss. Figure 3 demonstrates how we created positive and negative samples for every mini-batch. We conduct a grid search for a mini-batch size, choosing between 32 and 64, and the 8-bit Adam optimizer with a learning rate of 1e-4, 2e-5, or 5e-5. We fine-tune the model for 1 epoch.

We carry out experiments using the LACoS-BLOOM model with sizes ranging from 560m to 7b1 and adapter dimensions (r) of 1, 2, 4, 8, or 16. For each BLOOM model size, we retain the best checkpoint for the final evaluation. We utilize the same configuration as Sentence-BERT (SBERT) [9] with the softmax objective for the baseline, where the pre-trained model is "bert-base-multilingual-cased." The experiments were executed on a single GPU with Volta architecture and 32GB of memory.

To select the optimal model, we utilize the test dataset from SNLI and MNLI as our validation set. We aggregate the validation loss and standardize it to a common range. Figure 4 displays the number of adapters and the validation error at different BLOOM. The BLOOM 560m demonstrates the lowest validation error with four adapters for each module, whereas the BLOOM 7b1 exhibits the lowest validation error with one adapter per layer. This highlights that when the model size is small, it is necessary to enable more adapters, whereas when the model size is large, only a few adapters are sufficient.

3.2. Performance comparison

We assess the performance of the LACoS-BLOOM model on seven STS English tasks (STS12-16 [22], STS-B [23] and SICK-R [24]) and one multi-lingual STS task (xSTS [25]), all of which had not been included in the training process. The STS datasets provide labels ranging from 0 to 5, indicating the semantic relatedness of sentence pairs. We use the same evaluation metric as the baseline SBERT, which is the maximum Spearman's rank correlation among the cosine similarity, Manhattan-distance, Euclidean-distance, and dot-product similarity of sentence embeddings and the golden label datasets.

To evaluate the performance, we use two more metrics: STS-Avg. and xSTS-Avg. STS-Avg. is the average score among STS12-16, STS-B, and SICK-R, which are common benchmarks for

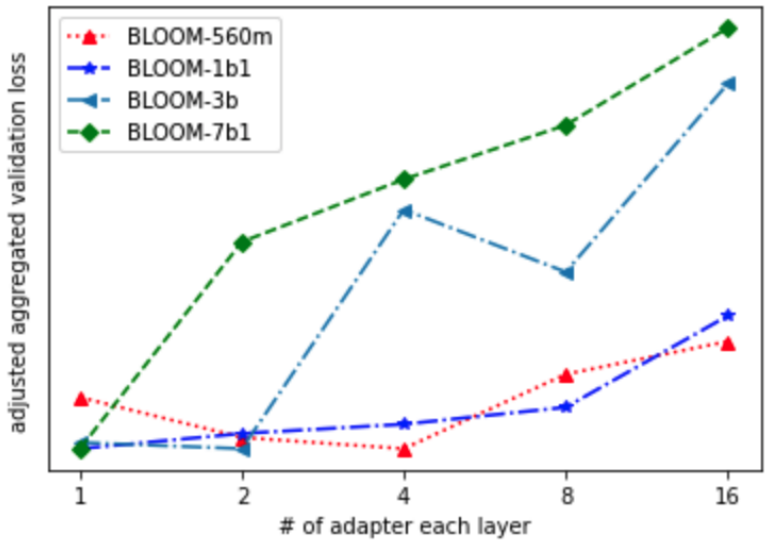

Figure 4: aggregated validation loss vs # of adapter for each layer

evaluating the performance of semantic text similarity models. xSTS-Avg. is the average score across all languages and was used to assess the cross-lingual performance of our model.

We evaluate the BLOOM models identified from Fig 4 on the STS tasks and present the correlation scores in Table 1. The scores increased as the model size increased, with LACoS-BLOOM 7b1 achieving the best performance. We use LACoS-BLOOM 7b1 to evaluate the English STS tasks and apply it to xSTS task. Our LACoS-BLOOM method improved the performance of both the English and multi-lingual STS task by at least 4+%. One observation is that SBERT's performance on multilingual tasks is not as good as it is on English tasks. This could be attributed to the fact that SBERT is a relatively small model, which may limit its ability to transfer knowledge from training to evaluation tasks that differ significantly.

Table 1
Sentence embedding performance on STS tasks; max(Spearman's rank correlation) (%); STS-Avg. is the average score among STS12-16, STS-B and SICK-R. xSTS-Avg. is the average score across all languages

training data	dataset / model size	STS12	STS13	STS14	STS15	STS16	STS-B	SICK-R	STS-Avg.	xSTS-Avg.
SNLI + MNLI	SBERT	59.46	56.49	53.58	61.15	57.42	57.69	61.94	58.42	52.30
	LACoS-BLOOM-560m	47.54	39.54	34.56	49.71	42.10	45.84	46.09	43.62	42.15
	LACoS-BLOOM-1b1	59.25	62.22	56.04	62.88	59.38	59.13	54.25	59.02	54.20
	LACoS-BLOOM-3b1	59.74	63.81	57.12	63.95	61.78	61.18	55.93	60.50	56.75
	LACoS-BLOOM-7b1	63.92	65.04	58.46	66.47	63.46	62.23	57.09	62.38	56.81
multi-NLI	SBERT	43.58	51.19	45.82	51.97	58.16	41.35	46.49	48.37	48.82
	LACoS-BLOOM-7b1	66.27	71.93	67.41	75.78	71.44	70.67	57.76	68.75	70.82

3.3. Ablation study

As BLOOM is an LLM, one advantage is its feasibility for transfer learning. Therefore, we perform zero-shot inference on BLOOM. On the other hand, the previous solution (i.e., simCSE) [15] showed that fine-tuning a full-size model with an MNR contrastive objective achieved the SOTA result on STS tasks. To make a fair comparison, we chose the BLOOM model with 1.1 billion parameters for zero-shot inference, LACoS-BLOOM, and full model fine-tuning, since this is the largest model size that can fit in a single GPU under simCSE setting.

The training data includes SNLI and MNLI for English tasks, and we evaluate the performance on STS tasks. The results are reported in Table 2. From the results, we find that LACoS-BLOOM outperforms the zero-shot solution. The performance of LACoS-BLOOM is comparable to the full model fine-tuning performance and the computational cost is much more lower.

Table 2
Ablation study, we trained the LACoS-BLOOM and full size model on SNLI and MNLI datasets, sentence embeddings were evaluated on STS tasks; max(Spearman's rank correlation) (%); STS-Avg. is the average score among STS12-16, STS-B and SICK-R. xSTS-Avg. is the average score across all languages

training data	dataset model setting	STS12	STS13	STS14	STS15	STS16	STS-B	SICK-R	STS-Avg.
SNLI	zero-shot learning	49.21	48.89	40.16	56.24	52.13	39.63	53.09	48.48
+	LACoS-BLOOM-1b1	59.25	62.22	56.04	62.88	59.38	59.13	54.25	59.02
MNLI	full size model finetune	69.71	73.14	68.64	76.94	76.17	70.44	68.30	71.90

4. Related work

4.1. Compressing LLM

Deep learning models, particularly transformer-based language models, have achieved SOTA results in NLP, computer vision, speech analysis and other tasks. However, these models can be computationally expensive, so various model compression methods, such as pruning, quantization, knowledge distillation, parameter sharing, tensor decomposition, and sub-quadratic Transformer-based methods, have been developed to reduce computational costs while maintaining performance [26, 27]. 8-bit quantization is a popular approach for optimization as it reduces both memory and computing requirements without the need to manipulate the architecture and can be used with machine learning frameworks and hardware toolchains. Different from previous work in applying quantization to their application, we use blockwise quantization and dequantization to save the footprint and maintain the perplexity score. As a result, such solution optimizes not just the network but the entire application (e.g., network bandwidth, inference latency and power consumption).

4.2. Parameter-efficient fine-tuning methods

In NLP, fine-tuning large pre-trained language models on downstream tasks is common practice but can be impractical as the model size and number of tasks increases. To address this, various

parameter-efficient transfer learning approaches have been proposed such as adapters [28], prefix-tuning [29] and LoRA [13]. The idea behind adapters is to fine-tune large models by only enabling a small subset of parameters on each transformer layer. Prefix-tuning keeps the model parameters frozen and only prepends a few examples to the task input while optimizing the objective based on the controllable prefix texts. LoRA utilizes the adapter approach, but instead of adding a subset of parameters, it enables a few low-intrinsic adapters in parallel with the attention module which does not increase inference latency. In this work, we are interested in LoRA because the design allows for flexibility in adding adapters anywhere [17], making it useful for scaling up to large language models for improved performance on specific tasks.

4.3. Siamese network with contrastive loss

Siamese architecture has been widely used in computer vision, NLP, and more. The goal is to learn a similarity function between two inputs. Recent work has shown that the Siamese architecture can boost performance with a self-learning objective in several natural language tasks (e.g., [9, 15, 30]). In this work, we propose to incorporate MNR objective. One advantage of the MNR is that it doesn't rely on labeled data and only considers ranking a set of similar items higher than a set of multiple dissimilar examples. This makes it more efficient in terms of computation and memory, and can lead to a more robust model that generalizes well to new examples.

5. Conclusion and future work

In this paper, we propose a parameter efficient fine-tuning method called LACoS-BLOOM for extracting multilingual text embeddings from a Large Language Model (LLM). We use 8-bit quantization to reduce the model footprint. We then improve the performance of LLM fine-tuning using LoRA, and further enhance semantic similarity using a Siamese network with MNR. Our solution can train 7.1 billion BLOOM end-to-end on a single GPU. On STS tasks, our method significantly outperforms the baseline as well as zero-shot LLM BLOOM. Our solution is able to scale up the LLM to 7.1 billion model. In the future, we plan to incorporate DeepSpeed with LACoS-BLOOM to efficiently scale up the training task to the full BLOOM.

References

[1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, arXiv preprint arXiv:2001.08361 (2020).

[2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, Advances in neural information processing systems 33 (2020) 1877–1901.

[3] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al., Palm: Scaling language modeling with pathways, arXiv preprint arXiv:2204.02311 (2022).

[4] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V. Korthikanti, et al., Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model, arXiv preprint arXiv:2201.11990 (2022).
[5] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training (2018).
[6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, OpenAI blog 1 (2019) 9.
[7] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805 (2018).
[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, arXiv preprint arXiv:1907.11692 (2019).
[9] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, arXiv preprint arXiv:1908.10084 (2019).
[10] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, et al., Bloom: A 176b-parameter open-access multilingual language model, arXiv preprint arXiv:2211.05100 (2022).
[11] H. Laurençon, L. Saulnier, T. Wang, C. Akiki, A. V. del Moral, T. Le Scao, L. Von Werra, C. Mou, E. G. Ponferrada, H. Nguyen, et al., The bigscience roots corpus: A 1.6 tb composite multilingual dataset, in: Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2022.
[12] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, Llm. int8 (): 8-bit matrix multiplication for transformers at scale, arXiv preprint arXiv:2208.07339 (2022).
[13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, Lora: Low-rank adaptation of large language models, arXiv preprint arXiv:2106.09685 (2021).
[14] T. Dettmers, M. Lewis, S. Shleifer, L. Zettlemoyer, 8-bit optimizers via block-wise quantization, arXiv preprint arXiv:2110.02861 (2021).
[15] T. Gao, X. Yao, D. Chen, Simcse: Simple contrastive learning of sentence embeddings, arXiv preprint arXiv:2104.08821 (2021).
[16] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy, et al., Text and code embeddings by contrastive pre-training, arXiv preprint arXiv:2201.10005 (2022).
[17] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, G. Neubig, Towards a unified view of parameter-efficient transfer learning, arXiv preprint arXiv:2110.04366 (2021).
[18] M. Henderson, R. Al-Rfou, B. Strope, Y. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos, R. Kurzweil, Efficient natural language response suggestion for smart reply. arxiv, Preprint posted online May 1 (2017).
[19] S. R. Bowman, G. Angeli, C. Potts, C. D. Manning, A large annotated corpus for learning natural language inference, arXiv preprint arXiv:1508.05326 (2015).
[20] A. Williams, N. Nangia, S. R. Bowman, A broad-coverage challenge corpus for sentence understanding through inference, arXiv preprint arXiv:1704.05426 (2017).
[21] A. Conneau, G. Lample, R. Rinott, A. Williams, S. R. Bowman, H. Schwenk, V. Stoyanov, Xnli: Evaluating cross-lingual sentence representations, arXiv preprint arXiv:1809.05053

(2018).
[22] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, A. Bordes, Supervised learning of universal sentence representations from natural language inference data, arXiv preprint arXiv:1705.02364 (2017).
[23] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, L. Specia, Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation, arXiv preprint arXiv:1708.00055 (2017).
[24] M. Marelli, S. Menini, M. Baroni, L. Bentivogli, R. Bernardi, R. Zamparelli, et al., A sick cure for the evaluation of compositional distributional semantic models., in: Lrec, Reykjavik, 2014, pp. 216–223.
[25] P. May, Machine translated multilingual sts benchmark dataset., 2021. URL: https://github.com/PhilipMay/stsb-multi-mt.
[26] P. Ganesh, Y. Chen, X. Lou, M. A. Khan, Y. Yang, H. Sajjad, P. Nakov, D. Chen, M. Winslett, Compressing large-scale transformer-based models: A case study on bert, Transactions of the Association for Computational Linguistics 9 (2021) 1061–1080.
[27] M. Gupta, P. Agrawal, Compression of deep learning models for text: A survey, ACM Transactions on Knowledge Discovery from Data (TKDD) 16 (2022) 1–55.
[28] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, S. Gelly, Parameter-efficient transfer learning for nlp, in: International Conference on Machine Learning, PMLR, 2019, pp. 2790–2799.
[29] X. L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, arXiv preprint arXiv:2101.00190 (2021).
[30] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, A. Overwijk, Approximate nearest neighbor negative contrastive learning for dense text retrieval, arXiv preprint arXiv:2007.00808 (2020).